

Partie 05 — Exercices

Répéter des opérations avec des boucles

Oskar Laverny

Table des matières

Exercice 1 — Boucle simple	1
Exercice 2 — Préallocation	1
Exercice 3 — Colonnes d'un tableau	1
Exercice 4 — <code>while</code>	1
Exercice 5 — Boucle vs vectorisation	2
Exercice 6 — Déboguer des boucles	2
a. Un seul élément est calculé	2
b. Le résultat précédent est écrasé	2
c. Une boucle qui ne s'arrête pas	2
Exercice 7 — Utiliser <code>next</code> et <code>break</code>	2
Exercice 8 — Mini-analyse avec <code>patients</code>	3
a. Résumer plusieurs colonnes	3
b. Construire une catégorie ligne par ligne	3
c. Comparer les écritures	3

Exercice 1 — Boucle simple

Pour :

```
x <- c(2, 5, 8, 11)
```

Écrivez une boucle qui affiche le carré de chaque valeur.

Exercice 2 — Préallocation

Créez un vecteur `res` contenant les carrés des entiers de 1 à 100 à l'aide d'une boucle `for` et d'une réallocation.

Vérifiez que `res` est identique à `(1:100)^2`.

Exercice 3 — Colonnes d'un tableau

Pour les colonnes `age`, `tas` et `score` de `patients`, utilisez une boucle pour calculer la moyenne de chaque colonne en ignorant les valeurs manquantes.

Stockez les trois moyennes dans un vecteur nommé.

Exercice 4 — `while`

On part de `x <- 1000`. Tant que `x` est supérieur à 10, divisez-le par 1.5.

1. affichez les valeurs successives ;
2. comptez le nombre d'itérations nécessaires.

Exercice 5 — Boucle vs vectorisation

Créez avec une boucle un vecteur valant "senior" lorsque `age >= 65` et "autre" sinon.

Puis écrivez une version vectorisée avec `ifelse()`.

Exercice 6 — Déboguer des boucles

Les boucles suivantes ne donnent pas le résultat attendu. Pour chacune :

1. expliquez le problème ;
2. corrigez le code ;
3. vérifiez le résultat obtenu.

a. Un seul élément est calculé

```
x <- c(2, 5, 8, 11)
res <- numeric(length(x))

for (i in length(x)) {
  res[i] <- x[i]^2
}
```

b. Le résultat précédent est écrasé

```
x <- c(2, 5, 8, 11)
res <- numeric(length(x))

for (i in seq_along(x)) {
  res <- x[i]^2
}
```

c. Une boucle qui ne s'arrête pas

Le code suivant provoquerait une boucle infinie. **Ne l'exécutez pas avant de l'avoir corrigé.**

```
x <- 100

while (x > 1) {
  print(x)
}
```

Exercice 7 — Utiliser `next` et `break`

On considère :

```
x <- c(4, NA, 9, -2, 16, 25, 36)
```

Écrivez une boucle qui :

1. ignore les valeurs manquantes et négatives avec `next` ;
2. affiche la racine carrée des autres valeurs ;
3. s'arrête avec `break` dès qu'elle rencontre une valeur strictement supérieure à 25.

Ajoutez un message permettant de comprendre pourquoi une valeur a été ignorée ou pourquoi la boucle s'est arrêtée.

Exercice 8 — Mini-analyse avec `patients`

Travaillez avec `patients.csv`.

a. Résumer plusieurs colonnes

1. créez `vars <- c("age", "tas", "score")` ;
2. préallouez deux vecteurs numériques nommés `moyennes` et `ecarts_types` ;
3. avec une boucle sur les noms de colonnes, calculez leur moyenne et leur écart-type en ignorant les valeurs manquantes ;
4. rassemblez les résultats dans un data frame comportant une ligne par variable.

b. Construire une catégorie ligne par ligne

Préallouez un vecteur de caractères de même longueur que `patients$score`. Remplissez-le avec une boucle selon les règles suivantes :

- "manquant" si le score est manquant ;
- "élevé" si le score est supérieur ou égal à 70 ;
- "habituel" sinon.

Ajoutez ce vecteur au tableau sous le nom `classe_score`, puis affichez les effectifs avec `count()` ou `table()`.

c. Comparer les écritures

Produisez la même classification avec une écriture vectorisée utilisant `ifelse()`. Vérifiez que les deux résultats sont identiques et indiquez en une ou deux phrases quelle écriture vous paraît la plus lisible ici.